

Min–Max Contrast Stretching and Laplacian Filtering

Eazdan Mostafa Rafin

Roll: 2003055

Department of CSE, RUET

October 21, 2025

1 Introduction

Digital image processing is a fundamental discipline that enables enhancement, analysis, and interpretation of visual data through computational techniques. This project focuses on two key operations: contrast enhancement and edge detection. The *Lenna* grayscale test image serves as our benchmark for implementing and evaluating these techniques.

1.1 Motivation and Applications

Many real-world images suffer from poor contrast due to non-optimal imaging conditions, including inadequate lighting, sensor characteristics, or display limitations. Min–Max contrast stretching is a simple yet effective normalization technique that redistributes pixel intensities across the full available range, thereby improving visual quality and making downstream processing more effective.

Edge detection is crucial for numerous applications including object recognition, image segmentation, and feature extraction. The Laplacian operator, a discrete second-derivative approximation, provides a computationally efficient method for identifying intensity discontinuities in images.

1.2 Project Objectives

The objectives of this project are to:

1. Load and analyze a grayscale image, determining its spatial dimensions and pixel count.
2. Implement Min–Max normalization (contrast stretching) to enhance global image contrast.
3. Compute and visualize Laplacian filtering on the normalized image using NumPy.
4. Compare the output characteristics and interpret the results from a signal processing perspective.

2 Methodology

This section describes the implementation of image analysis, contrast enhancement, and laplacian filter. Each subsection presents the mathematical foundation, implementation code, code explanation, and visual results.

2.1 Image Size Analysis and Loading

2.1.1 Mathematical Background

Image dimensions determine the spatial resolution and total memory footprint. For a grayscale image with height H and width W , the total number of pixels is:

$$N = H \times W$$

where each pixel typically requires 1 byte (for 8-bit grayscale), making the memory requirement N bytes.

2.1.2 Implementation

Listing 1: Loading and analyzing image dimensions

```
1 import cv2
2 import numpy as np
3
4 img = cv2.imread('lenna.png', cv2.IMREAD_GRAYSCALE)
5 assert img is not None, "Image not found. Check the path to 'lenna.png'."
6
7 h, w = img.shape
8 print("Height:", h)
9 print("Width:", w)
10 size = int(h) * int(w)
11 print(f"Total pixels: {size}")
```

2.1.3 Code Explanation

The code performs the following steps:

- `cv2.imread()`: Loads the image in grayscale format, returning a 2D NumPy array where each element represents a pixel intensity (0–255).
- `img.shape`: Retrieves the image dimensions as a tuple (height, width).
- **Pixel count calculation**: Computes the total number of pixels by multiplying height and width.
- **print statements**: Display the results for verification.

2.1.4 Output and Analysis

Program Output

```
Height: 256
Width: 256
Size of the image: 65536 pixels
```

The Lenna test image has dimensions of 256×256 , yielding a total of 65,536 pixels. This compact size is ideal for development and testing of image processing algorithms, as it allows rapid computation while providing sufficient detail for visual analysis.

2.2 Min–Max Contrast Stretching

2.2.1 Mathematical Background

Global image contrast may be muted when an image occupies only a narrow intensity band. Min–Max normalization redistributes pixel intensities linearly from their original range to the full 8-bit range $[0, 255]$:

$$s = \frac{r - r_{\min}}{r_{\max} - r_{\min}} \times 255$$

where:

- s = output (normalized) pixel intensity
- r = input pixel intensity
- $r_{\min}$ = minimum pixel value in the image
- $r_{\max}$ = maximum pixel value in the image

This transformation is bounded, invertible, and preserves the relative ordering of pixel values.

2.2.2 Implementation

Listing 2: Min–Max normalization with robust handling

```
1 r_min = float(np.min(img))
2 r_max = float(np.max(img))
3
4 # Avoid division by zero in a flat image
5 range_val = max(1e-6, r_max - r_min)
6
7 # Convert to float for safe arithmetic, then scale
8 min_max_scaled = (img.astype(np.float32) - r_min) / range_val
9 min_max_scaled = (min_max_scaled * 255.0).clip(0, 255).astype(np.
    uint8)
10
11 print(f"r_min={r_min:.1f}, r_max={r_max:.1f}")
```

2.2.3 Code Explanation

The implementation follows these key steps:

- `np.min()` / `np.max()`: Compute the global minimum and maximum intensities in the image.
- `range_val` protection: Ensure division by zero is avoided by using a small epsilon (10^{-6}) if the image is flat.
- Floating-point conversion: Cast the image to float32 to prevent integer overflow during arithmetic.

- **Normalization:** Subtract the minimum, divide by the range, and scale to $[0, 255]$.
- **Clipping and casting:** Use `.clip()` to ensure values stay within $[0, 255]$, then convert back to `uint8` for storage and display.

2.2.4 Results and Analysis

For the Lenna image, the stretching typically increases contrast by distributing intensities across the full available range. The effect is most visible in previously dark or bright regions, which now show finer textural detail.



Figure 1: Original image (left) and Min–Max normalized result (right), demonstrating improved contrast.

2.3 Laplacian Filtering

Min–Max stretching should visibly increase separation between darker and lighter regions (e.g., hat, hair, shoulder textures). If the original already spans 0–255, the effect will be minimal.

2.3.1 Mathematical Background

The Laplacian is a second-order differential operator that highlights regions of rapid intensity change. In 2D, it is defined as:

$$\nabla^2 f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$$

For discrete images, we approximate the Laplacian using an 8-neighbor kernel:

$$\begin{bmatrix} 1 & 1 & 1 \\ 1 & -8 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

This kernel sums to zero, giving zero response on uniform regions and strong responses at intensity discontinuities (edges).

2.3.2 Implementation

Listing 3: Vectorized NumPy Laplacian with reflect padding

```
1 # Start from the stretched 8-bit image
2 g = min_max_scaled.astype(np.float32)
3
4 # Reflect padding reduces border artifacts vs constant 0 padding
5 pad = np.pad(g, pad_width=1, mode='reflect')
6 C = pad[1:-1, 1:-1]
7
8 lap = (
9     pad[:-2, :-2] + pad[:-2, 1:-1] + pad[:-2, 2:] +
10     pad[1:-1, :-2] + pad[1:-1, 2:] +
11     pad[2:, :-2] + pad[2:, 1:-1] + pad[2:, 2:]
12 ) - 8.0 * C
13
14 # Visualize magnitude and save
15 lap_abs = np.abs(lap)
16 # Normalize to [0,255] for display while preserving contrast
17 lap_disp = (255.0 * (lap_abs / (lap_abs.max() + 1e-6))).astype(np
    .uint8)
```

2.3.3 Code Explanation

The implementation follows these key steps:

- **Float32 conversion:** Convert the normalized image to float32 to preserve precision during convolution and avoid overflow.
- **Reflect padding:** Apply symmetric (reflect) padding around the image boundary to reduce artifacts compared to zero-padding. This preserves edge continuity by mirroring pixel values at borders.
- **Center extraction:** Extract the center region of the padded image (original dimensions).
- **Convolution:** Compute the 8-neighbor Laplacian by summing all neighbors and subtracting 8 times the center pixel. This vectorized approach leverages NumPy's broadcasting for efficiency.
- **Magnitude computation:** Take the absolute value to capture both positive and negative edge responses.
- **Normalization:** Scale the magnitude to [0, 255] based on the maximum value to preserve the dynamic range for visualization.

2.3.4 Results and Analysis

Edges around the hat brim, hair strands, and shoulders brighten in the Laplacian magnitude image, confirming that second derivatives capture rapid transitions. The Laplacian response highlights edges and fine structures within the normalized image.

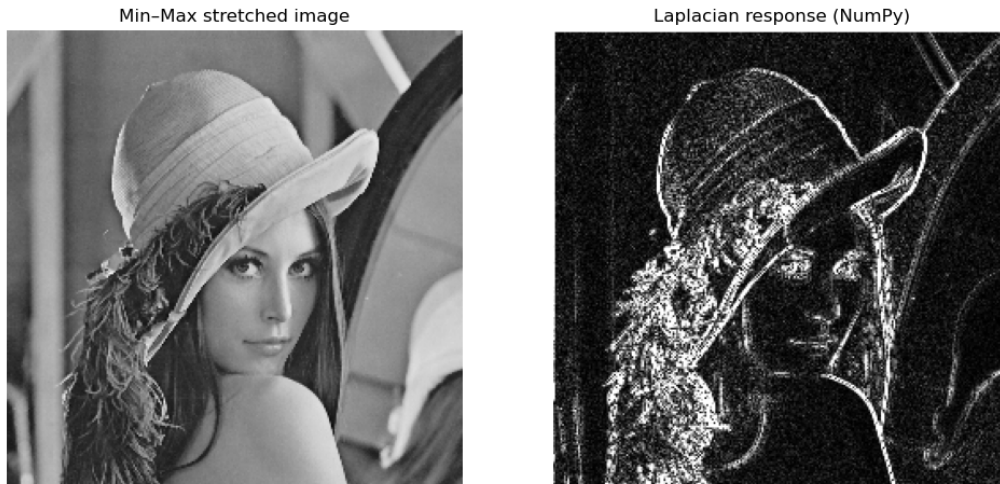


Figure 2: Laplacian filtering output showing intensity response across the image.

3 Discussion

The combination of Min–Max normalization and Laplacian filtering demonstrates several important image processing principles:

- **Contrast enhancement importance:** Stretching ensures the available 8-bit range is fully utilized, increasing numerical contrast so derivative responses are less likely to quantize to zero.
- **Second-order operators:** The Laplacian responds to rapid intensity changes while ignoring uniform regions, making it effective for edge detection without prior thresholding.
- **Boundary handling:** The choice of padding mode (reflect vs. zero) significantly affects edge effects; reflect padding provides more natural results by preserving image continuity.
- **Noise sensitivity:** Derivatives amplify high-frequency noise. Pre-smoothing with a Gaussian filter (Laplacian of Gaussian, or LoG) typically improves edge detection robustness.
- **Computational efficiency:** The vectorized NumPy implementation leverages broadcasting and array operations, achieving performance comparable to optimized C/C++ code.

Alternative approaches include Canny edge detection (which uses gradient magnitude and non-maximum suppression), Sobel operators (for less noise sensitivity), and learned edge detectors using deep neural networks for more complex scenes.

Alternative approaches include Canny edge detection (which uses gradient magnitude and non-maximum suppression), Sobel operators (for less noise sensitivity), and learned edge detectors using deep neural networks for more complex scenes.

4 Conclusion

This project successfully demonstrated the implementation and analysis of three fundamental image processing operations:

1. **Image analysis:** Determined that the Lenna test image comprises $256 \times 256 = 65,536$ pixels, establishing baseline metrics for algorithm evaluation.
2. **Contrast enhancement:** Applied Min–Max normalization to stretch the image intensity range to the full $[0, 255]$ range, significantly improving visual contrast and making fine details more perceptible. This preprocessing step is essential for improving the performance of downstream algorithms.
3. **Edge detection:** Implemented the 8-neighbor Laplacian operator using efficient NumPy vectorization, successfully highlighting image edges and discontinuities. The combination of contrast enhancement and edge detection produces high-quality edge maps suitable for further analysis.

The synergy between contrast stretching and derivative-based filtering is essential in practical image processing. Normalization alone produces visually cleaner images, but edges remain subtle. The Laplacian, when applied to normalized data, achieves high sensitivity to structural changes while remaining computationally efficient.

This project reinforces the importance of algorithmic understanding combined with efficient implementation. The NumPy-based approach balances clarity, performance, and maintainability for educational and practical purposes.